Azure SQL Elastic Pool Production Deployment Runbook

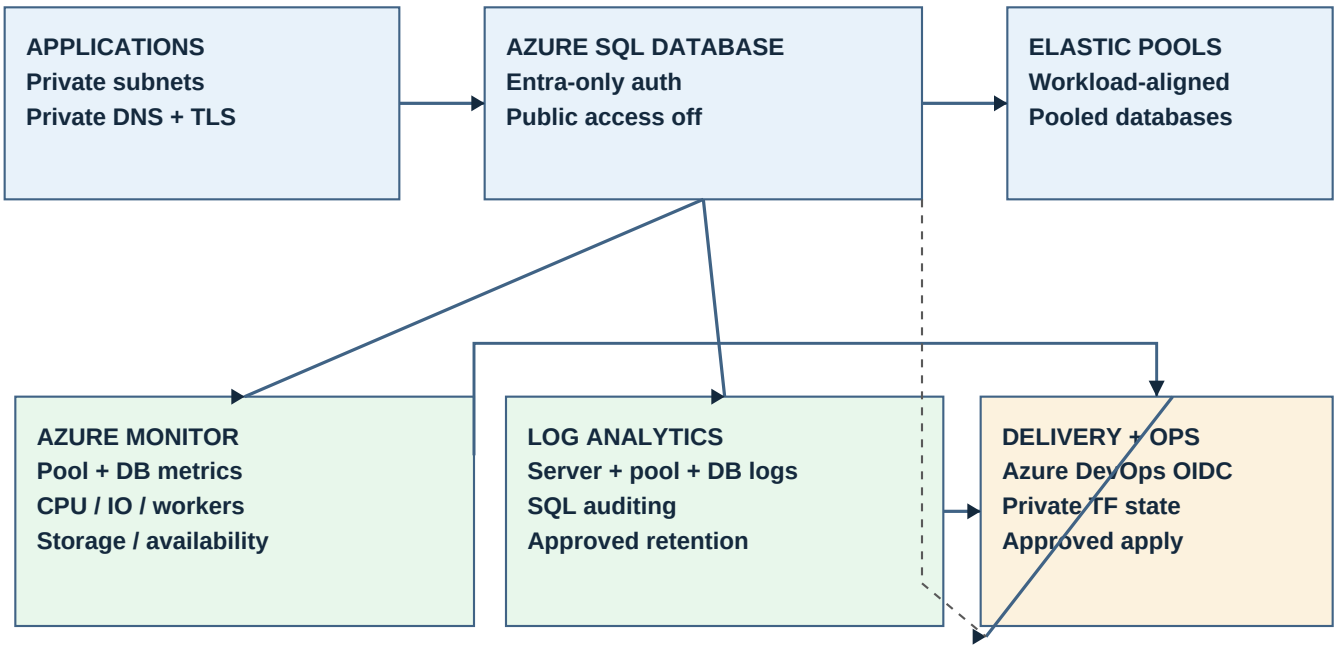
Terraform, Azure DevOps, private networking, monitoring, operations, and recovery

Reference scale: 10+ elastic pools / 100+ databases

Azure SQL Database - operational reference; validate values for your environment.

Project: azure-sql-elastic-pool-production-runbook

PRODUCTION REFERENCE ARCHITECTURE



Applications reach the Entra-only SQL server over Private Link and private DNS. Terraform is delivered by Azure DevOps workload identity; its private Blob state backend uses lease locking. Azure Monitor metrics, server auditing, and resource diagnostics feed Log Analytics and staffed alerts. A separately tested recovery region supports the approved continuity plan.

Reference only, not a deployed environment. Confirm topology, regional redundancy, product support, SKU, cost, and RPO/RTO with service owners.

Azure SQL elastic pool production deployment runbook

Status: Production-oriented implementation template. Not a substitute for workload-specific design review, change approval, capacity testing, or recovery evidence. Do not deploy example values unchanged.

Service scope: Azure SQL Database elastic pools, vCore model, one primary region. Azure SQL Managed Instance and SQL Server on VMs are out of scope.

Deployment entry point: terraform/production
Delivery pipeline: azure-pipelines.yml
Architecture: Reference architecture

1. Prerequisite readiness gates

Do not start a production apply until every item below is verified and the responsible owner has recorded the evidence in the change request. Microsoft-hosted agents can validate code, but private state access requires the configured deployment agent to reach the state account's private endpoint.

Gate | Required prerequisite and evidence | Owner
Scope and ownership | Approved service owner, database owner, operational/on-call contact, environment, data classification, cost center, and change ticket. | Service owner
Azure subscription | Correct tenant, subscription, management group/policy, resource group naming, approved primary region, and subscription/resource-group quotas. Confirm Microsoft.Sql, Microsoft.Network, Microsoft.Insights, and Microsoft.OperationalInsights providers are registered, deployer access is scoped, and budget is approved. | Cloud platform
Capacity and SKU | Database inventory; representative workload measurements; approved pool placement, vCore tier/hardware family, capacity, storage, per-DB bounds, zone redundancy, and cost estimate. Confirm SKU availability and regional quota. | DBA + workload owner
Identity | Entra administrator group/object ID and tenant; Microsoft Entra-only SQL sign-in; app managed identities or approved groups; no shared SQL admin password. | Identity + DBA
Network | Existing VNet, approved Private Endpoint subnet, address capacity, routing/NSG review, and egress to Azure Resource Manager and the private state backend. | Network
Private DNS | Existing privatelink.database.windows.net zone, correct VNet link, permissions for Private Endpoint DNS zone group, and tested resolution from workload subnets. | Network/DNS
State backend | Dedicated Azure Storage account/container, private endpoint and DNS, TLS 1.2+, public blob access disabled, shared-key access disabled, soft delete/versioning per policy, resource locks/backup where approved, and Azure Blob lease locking. | IaC platform
Azure DevOps | Project/repository, protected main, pull-request reviewers, pipeline authorization, approved self-hosted Linux deployment pool, and required Terraform installer extension/version. | DevOps
Workload identity | Azure Resource Manager service connection using workload identity federation (OIDC), explicit tenant/subscription, and least-privilege role assignments for the app resource group, subnet, DNS zone, and state container. | Cloud platform
Production approval | Protected sql-prod Azure DevOps Environment with named approvers/checks, restricted pipeline permissions, and an agreed rollback/stop authority. | Change manager
Observability | Approved Log Analytics workspace region/retention/access mode, metric alert thresholds, staffed action group recipient, incident routing, dashboards, and diagnostic retention/cost review. | SRE/observability
Security/compliance | Network and identity threat review, Azure Policy and Defender requirements, auditing/diagnostics, encryption requirements, data residency, vulnerability management, and deletion/restore controls. | Security/compliance
Resilience | Signed RPO/RTO, restore/failover design, secondary region and quota, database/pool mapping, application retry and DNS/listener plan, and scheduled test owner. | Service owner + DBA
Change safety | Nonproduction deployment and application test complete; reviewed Terraform diff/plan; backup/restore evidence; maintenance window; communications and escalation path. | Change approver

1.1 Resolve failed prerequisites before continuing

Symptom | Resolution; do not bypass the control
Wrong tenant/subscription, no subscription, or deploy denied | Stop. Use an approved identity, select the intended tenant and subscription, request scoped RBAC from the subscription owner, and rerun the identity checks. Do not widen permissions to Owner as a workaround.
SKU unavailable or quota insufficient | Have the capacity owner choose a supported tier/family/region or request quota through the subscription process. Recalculate cost and recovery capacity; update reviewed Terraform values. Do not reduce workload headroom without approval.
No measured inventory or workload baseline | Collect representative Azure SQL metrics and database ownership/criticality. Run a nonproduction load test where historical telemetry is inadequate. Do not infer pool size from database count alone.
No Entra administrator/group | Ask the Entra administrator to create or approve a managed operational group, verify its object ID in the correct tenant, and assign group membership through access review. Do not substitute a personal account or SQL password.
Private Endpoint subnet is invalid/full or lacks permissions | Network owner selects a supported subnet, reviews IP capacity/NSG/routes, and grants only the necessary subnet join/write permission. Re-read the exact resource ID; never create a duplicate/shared-network replacement without approval.
Private DNS lookup returns public/no address | Network/DNS owner verifies the exact private zone, VNet link, endpoint zone group, custom DNS forwarding, and resolver path. Test from the application subnet. Do not enable broad public SQL access to make the test pass.
State backend cannot be reached or authenticated | Check private agent routing/DNS, storage firewall/private endpoint, tenant/subscription, workload-identity federation, Storage Blob Data Contributor scoped to the state container, and AzureRM backend use_azuread_auth/OIDC settings. Do not use account keys or SAS tokens.

Azure DevOps cannot acquire OIDC token | Recreate/repair the service connection with workload identity federation, confirm the Azure DevOps issuer/subject/audience federation credential matches the connection, authorize the pipeline, and validate the correct tenant. Do not paste a client secret into YAML.
Pipeline identity lacks a required action | Use the failed Azure activity to identify the resource and permission. Have the owner add the narrow role at the target RG/subnet/DNS zone/container, then rerun a fresh plan. Do not grant Owner/Contributor at tenant or subscription scope just to silence an error.
Agent cannot reach Terraform downloads or Azure APIs | Have DevOps/network allow the approved artifact/provider registries and Azure endpoints, or use an approved internal mirror. Verify TLS inspection and agent clock/certificates. Do not disable certificate validation.
Pipeline has no protected environment/approvers | DevOps creates and secures the environment, restricts pipeline permissions, configures required approvers and branch checks, and tests a nonproduction gate. Do not remove approval to meet a deployment date.
Alert recipient/workspace policy is not approved | Observability team provides the staffed action-group target, workspace, region, access mode, retention and cost. Verify a test notification and alert ownership before accepting the service.
Restore/failover objective or evidence is missing | Service owner and DBA agree RPO/RTO and test a restore and regional recovery path with application owners. Record measured timings and data loss. No production go-live until the evidence matches the objective.
Terraform validate/plan fails | Read the first specific error; confirm Terraform version, provider lock file, required variables, region/SKU, IDs, permissions, backend reachability, and variable file. Fix the cause in a reviewed change. Do not suppress validation or apply a stale plan.

2. Operator and change controls

1. Identify the change owner, implementer, reviewer, approver, incident commander, application test contact, and rollback decision owner.
2. Attach the completed prerequisite evidence, capacity/cost review, security review, network approval, nonproduction test results, and recovery evidence to the change.
3. Freeze unrelated changes to the Terraform state key during the release. Terraform's Blob lease serializes state operations, but it does not replace a change freeze or review.
4. Review the source diff and the complete Terraform plan. Confirm all create/update/delete/replace actions and data-retention effects. If the plan is unexpected, stop; do not approve the production environment.
5. Use the Azure DevOps sql-prod Environment approval only after the exact pipeline plan artifact and change record have been reviewed.
6. Stop on any unplanned data-plane, network, identity, deletion, replacement, or destructive action. The starter pipeline has no destroy path.

3. Workstation and access prerequisites

3.1 Install and verify local tooling

Install approved, current versions of Git, Terraform CLI (same version as terraformVersion in azure-pipelines.yml), Node.js 18 or later, and Azure CLI. Install the Azure CLI Bicep extension only if validating the retained Bicep reference. Use organization-approved package sources and verify tool versions.

PowerShell:

```
git --version
terraform version
node --version
az version
```

If a command is not found, install it from the vendor/approved package repository, open a new terminal so PATH is refreshed, and rerun all four checks. If Terraform reports the wrong version, install the pinned version and align the pipeline version deliberately; regenerate and review .terraform.lock.hcl only when provider upgrades are approved.

3.2 Verify Azure identity and target

Use a named, approved operator identity; MFA and conditional access remain in force. Authenticate and verify the tenant before any change:

```
az login --tenant "<tenant-id>"
az account list --output table
az account set --subscription "<subscription-id>"
az account show --query "{tenant:tenantId,subscription:id,name:name,user:user.name}" --output json
```

Compare tenant and subscription IDs with the change record. If no account is returned, request access from the tenant/subscription owner. If the wrong subscription is active, stop and set the approved ID explicitly; do not rely on a remembered CLI context.

Local interactive login is for inspection and nonproduction testing. The production pipeline uses its federated identity and should not rely on a developer's CLI session.

4. Platform preparation

4.1 Network and DNS

Ask the network owner to provide and verify:

- VNet and subnet resource IDs in the SQL server's region or an approved supported topology. The endpoint subnet must support Private Endpoints and have available IP addresses.
- Required subnet policy, route table, NSG, DNS forwarding, and firewall controls for the endpoint and applications.
- The existing privatelink.database.windows.net Private DNS zone ID and a link to each approved client VNet. The Terraform endpoint zone group attaches the private endpoint to the provided zone; it does not create the zone/link.
- Private DNS forwarding/resolver behavior for corporate and build networks. From an approved workload host, verify the server FQDN resolves to the endpoint's private address and that TCP 1433 is reachable only where authorized.
- A self-hosted Azure DevOps Linux agent with outbound Azure Resource Manager access and network/DNS access to the private state backend. For SQL application smoke tests, use a separate approved workload runner inside the application network.

Do not enable SQL public network access or add an unrestricted firewall rule to compensate for missing routing, DNS, or build-agent connectivity.

4.2 Terraform state backend

Create the state resource group, dedicated StorageV2 account and blob container before running the application Terraform. Use the organization's approved region/redundancy, private endpoint, private DNS zone privatelink.blob.core.windows.net and VNet link, network restrictions, TLS 1.2 minimum, disabled public blob access, disabled shared-key access, Microsoft Entra data-plane authorization, soft delete/versioning, and state backup/retention policy. Enable Blob versioning and deletion recovery as required by policy. Restrict state and historical versions because Terraform state can contain sensitive resource metadata.

For a dedicated state resource group, the following is an example bootstrap sequence. Run it only with the approved platform/bootstrap identity and adapt the region, redundancy, names, tags, and retention to organizational policy. The Storage account is created with public network access disabled; the network team must attach its private endpoint and DNS before any blob data-plane operation. Do not temporarily open public access to get around a missing private route.

```
$stateResourceGroup = "rg-tfstate-prod"
$stateAccount = "<globally-unique-state-storage-name>"
$stateLocation = "<approved-state-region>"
$stateContainer = "tfstate"
az group create --name $stateResourceGroup --location $stateLocation
az storage account create `
--name $stateAccount `
```

```
--resource-group $stateResourceGroup `
--location $stateLocation `
--sku Standard_GRS `
--kind StorageV2 `
--https-only true `
--min-tls-version TLS1_2 `
--allow-blob-public-access false `
--allow-shared-key-access false `
--public-network-access Disabled
```

After the network team confirms the Storage private endpoint and DNS are reachable from the private deployment agent, enable recovery controls and create the container using Entra data-plane authentication:

```
$subscriptionId = az account show --query id --output tsv
$storageScope =
"/subscriptions/$subscriptionId/resourceGroups/$stateResourceGroup/providers/Microsoft.Storage/storageAccounts
/$stateAccount"
# A role administrator assigns this to the approved bootstrap identity.
az role assignment create `
--assignee-object-id "<bootstrap-principal-object-id>" `
--assignee-principal-type User `
--role "Storage Blob Data Contributor" `
--scope $storageScope
# Allow for RBAC propagation before using the blob data plane.
az storage account blob-service-properties update `
--account-name $stateAccount `
--resource-group $stateResourceGroup `
--enable-versioning true `
--enable-delete-retention true `
--delete-retention-days 30 `
--enable-container-delete-retention true `
--container-delete-retention-days 30
az storage container create `
--account-name $stateAccount `
--name $stateContainer `
--auth-mode login
az storage container show `
--account-name $stateAccount `
--name $stateContainer `
--auth-mode login `
--output table
```

The bootstrap identity needs approved management-plane rights to create the resource group/account and storage data-plane rights to create the container. Have the role administrator grant the pipeline identity **Storage Blob Data Contributor** at the existing container scope, not by sharing a key:

```
$subscriptionId = az account show --query id --output tsv
$containerScope = "$storageScope/blobServices/default/containers/$stateContainer"
az role assignment create `
--assignee-object-id "<azure-devops-service-principal-object-id>" `
--assignee-principal-type ServicePrincipal `
--role "Storage Blob Data Contributor" `
--scope $containerScope
```

Follow the platform's lock, monitoring, soft-delete retention, and state restoration policies. If provider registration is missing, ask the subscription owner to approve/register the listed providers; do not use broad subscription permissions as a workaround. Test az storage container show from the deployment agent, then enter the storage names and stable key in the protected Azure DevOps variable group.

Use a dedicated, stable key such as azure-sql/prod.tfstate. The AzureRM backend uses blob leases for state locking. Grant the pipeline identity Storage Blob Data Contributor at the state container (or the narrowest supported scope), and authorize state-management actions separately. Never commit state, plan files, access keys, SAS tokens, credentials, or state backups.

If the storage account is private-only, the plan and apply agent must be able to resolve and reach its private endpoint before pipeline use. Test access from that agent; an Azure-hosted Microsoft-managed runner does not automatically have access to a private VNet. Use the organization's private deployment pool.

4.3 Identity and Azure DevOps

1. Create an Azure DevOps project/repository and protect main: require reviewed pull requests, build validation, no direct pushes, and restricted pipeline-edit permissions.
2. Install the approved Microsoft DevLabs Terraform extension to provide TerraformInstaller@1. Pin the tested Terraform CLI version in azure-pipelines.yml; maintain the Azure CLI task on the Microsoft-hosted task catalog.
3. Create an Azure Resource Manager service connection with **workload identity federation**, not a client secret. Use the exact tenant and subscription. Configure and validate the generated federation credential; authorize only this pipeline. Add a server-side Branch control check to the service connection that allows only protected refs/heads/main and fails closed for unknown protection status. Do not rely on YAML stage conditions alone to protect the connection from pull-request code.
4. Assign the service identity only the resource-group rights needed to create SQL, monitoring, and private endpoint resources, plus the narrowly scoped subnet and Private DNS permissions required for endpoint creation. Grant state Blob data access on the state container. Do not grant tenant-wide directory roles or subscription Owner for routine deployment.
5. Create an approved, private-connected Linux deployment agent pool. Install Git, Bash, Azure CLI, the pinned Terraform CLI or Terraform extension support, and required CA certificates. Keep agent patches/current images and restrict who can administer the pool.
6. Create the Azure DevOps variable group named sql-elastic-pool-prod; add these non-secret values and authorize only the deployment pipeline:

Variable | Example/meaning
azureServiceConnection | Exact workload-identity service connection name.
deploymentAgentPool | Private Linux agent pool name.
azureSubscriptionId | Approved subscription GUID.
tenantId | Approved Microsoft Entra tenant GUID.
tfstateResourceGroup | State storage resource group name.
tfstateStorageAccount | Globally unique state StorageV2 account name.
tfstateContainer | Existing private blob container name.
tfstateKey | Stable state key, e.g. azure-sql/prod.tfstate.
tfVarsFile | environments/prod.tfvars (relative to terraform/production).

These values are identifiers, not credentials. Do not add client secrets, storage keys, SQL passwords, SAS tokens, or OIDC tokens. Do not make the variable group broadly accessible.

7. Create the Azure DevOps Environment sql-prod, restrict pipeline access, and configure required named approvers/checks, branch control, and any organization-required change-window checks. Verify that the approval is evaluated before apply.
8. Configure protected repository policies and pipeline resource permissions. A pull request can run offline validation but must not receive the production service connection or state access. Verify the service connection's server-side Branch control check rejects a test run from a feature branch; YAML conditions alone are not an authorization boundary.
9. In Azure DevOps, create a pipeline from this repository and select /azure-pipelines.yml. Authorize the variable group, workload-identity service connection, and private deployment-agent pool for this pipeline only. Run the initial PR validation first; the main pipeline must show a saved plan and stop for the protected sql-prod approval before any apply.

4.4 Workload resource group and Azure providers

Terraform expects the dedicated workload resource group to exist. The platform team creates it under the approved landing-zone policy, tags, budget, locks,

and region. If policy allows the bootstrap operator to create it directly:

```
$resourceGroup = "rg-sql-prod"
$location = "<approved-sql-region>"
az group create `
--name $resourceGroup `
--location $location `
--tags environment=production serviceOwner="<owner>" costCenter="<cost-center>"
dataClassification="<classification>"
az group show --name $resourceGroup --output table
```

Replace every placeholder and confirm the policy assignments, location, ownership tags, budget, and lock behavior with the landing-zone owner. Do not create a second group if the platform team has already provided one.

Check that required resource providers are registered in the target subscription:

```
$namespaces = @(
"Microsoft.Sql",
"Microsoft.Network",
"Microsoft.Insights",
"Microsoft.OperationalInsights",
"Microsoft.Storage"
)
foreach ($namespace in $namespaces) {
az provider show --namespace $namespace --query registrationState --output tsv
}
```

If any provider is not Registered, have the subscription owner review and register that namespace (az provider register --namespace <namespace>), then wait for registration to complete before planning. The Azure DevOps identity needs the approved resource-group permissions for SQL/monitoring resources and separate narrow subnet/Private DNS permissions for the Private Endpoint. It must not receive permission to create role assignments; the platform identity pre-assigns the required roles.

5. Capacity, configuration, and code review

5.1 Select pool and database configuration

Start from the inventory and measurements, not the example values. Set:

- location, resource group, globally unique SQL server name, Entra administrator login/object ID, and approved resource tags. Provider tenant and subscription are taken from the verified federated/CLI identity.
- private_endpoint_subnet_id and private_dns_zone_id from the network owner.
- pools: one entry per workload-aligned pool; supported vCore SKU/tier, capacity, hardware family, license model, storage, per-database minimum and maximum, and zone redundancy.
- databases: database-to-pool assignment, storage limit and short-term-retention days, plus approved backup storage redundancy (Geo, GeoZone, Local, or Zone).
- workspace_retention_in_days, action group email, and measured alert thresholds for pool CPU and data I/O.

The values in terraform.tfvars.example are illustrative only. Confirm that each SKU/family and feature is available in the selected region and supported for every database. Validate aggregate pool capacity, per-database limits, storage and workload concurrency. BasePrice requires approved Azure Hybrid Benefit eligibility; zone-redundant vCore pools require a supported BusinessCritical SKU/region. Get written approval for exceptions.

5.2 Create the production variable file

Create the environment directory and copy the example:

```
New-Item -ItemType Directory -Force terraform/production/environments
Copy-Item terraform/production/terraform.tfvars.example `
terraform/production/environments/prod.tfvars
```

Replace all example values with reviewed, non-secret production configuration. Do not add passwords, keys, tokens, private certificates, or credentials; Entra-only authentication eliminates the need for a SQL admin password. Terraform configuration values can expose infrastructure identifiers. Review repository access and secret-scanning rules before committing the environment file. The production pipeline intentionally fails until the approved file exists.

From a network with approved Terraform Registry access, initialize the provider and review/commit terraform/production/.terraform.lock.hcl. Ensure it contains checksums for the deployment agent's Linux platform (and Windows if used locally). The provider version is pinned in versions.tf; the lock file records verified package checksums. Do not remove it to work around a provider error.

6. Validate and plan

6.1 Credential-free checks

Run from the repository root:

```
terraform -chdir=terraform/production fmt -check -recursive
terraform -chdir=terraform/production init -backend=false
terraform -chdir=terraform/production validate
node scripts/build-pdf.mjs
git diff --check
```

These checks need no Azure credentials, but provider initialization requires approved network access to the Terraform Registry or the organization's provider mirror.

If fmt fails, run terraform -chdir=terraform/production fmt -recursive, review the result, and rerun all checks. If provider installation fails, verify the approved provider registry/mirror and TLS path; do not disable TLS checks. If validation reports a missing variable or invalid region/SKU, repair the reviewed configuration and rerun. Do not proceed with a failed check.

6.2 Authenticated nonproduction plan

First deploy and test in a separate nonproduction resource group, using its own reviewed variable file (copy the example to environments/nonprod.tfvars), backend key, and protected identity. Authenticate through an approved identity and verify the target subscription. Run terraform init against the dedicated remote backend. With Azure CLI authentication and the nonproduction values substituted, initialize the AzureRM backend as follows:

```
$env:ARM_SUBSCRIPTION_ID = az account show --query id --output tsv
$env:ARM_TENANT_ID = az account show --query tenantId --output tsv
# Confirm both values match the approved tenant and subscription in the change.
terraform -chdir=terraform/production init -reconfigure `
-backend-config="resource_group_name=<state-resource-group>" `
-backend-config="storage_account_name=<state-storage-account>" `
-backend-config="container_name=<state-container>" `
-backend-config="key=azure-sql/nonprod.tfstate" `
-backend-config="subscription_id=$env:ARM_SUBSCRIPTION_ID" `
-backend-config="tenant_id=$env:ARM_TENANT_ID" `
-backend-config="use_azuread_auth=true"
```

Then create the saved plan:

```
terraform -chdir=terraform/production plan `
-input=false `
-lock-timeout=5m `
-parallelism=5 `
-var-file=environments/nonprod.tfvars `
-out=nonprod.tfplan
```

Review the plan, test DNS/private connectivity, application authentication, diagnostic ingestion, action-group delivery, and realistic workload. Treat plan files as sensitive; store only in an access-controlled pipeline artifact

or delete a local file after approved use. Do not apply a plan produced from a different commit, backend, variable file, identity, or environment.
The pipeline limits apply/plan concurrency to five operations to reduce resource-provider throttling during fleet-scale deployment. Change that limit only after observing provider quotas and representative deployment timing.

6.3 Production plan in Azure DevOps

The pipeline runs credential-free formatting/validation for pull requests. After reviewed code merges to protected main, it uses the federated service connection to initialize the AzureRM backend and creates an immutable tfplan artifact. Reviewers compare the artifact to the merged Terraform, variable file, ticket, resource IDs, cost, and expected actions. An unexpected replacement or deletion blocks approval.

The production plan includes the full parameter values in Terraform's execution context. Limit artifact access/retention to authorized operators. Never publish it as a public artifact or attach it to an unrestricted work item. The apply stage downloads and applies the exact saved artifact; it does not silently re-plan.

7. Production apply

1. Confirm prerequisite gates, nonproduction acceptance, active change window, reviewer sign-off, exact main commit, plan artifact, state key, and target subscription.
2. Verify the sql-prod environment displays the required approvers and checks.
3. Approve only when planned resources, replacements, action group, SKU, private endpoint, diagnostic settings, retention, and estimated costs match the approved change.
4. The deployment job applies the saved plan. Watch the Azure DevOps job and Azure Activity Log. Do not run a second concurrent pipeline on the same key.
5. Stop on apply errors or drift. Preserve logs and plan identifiers; diagnose the failing Azure API, permission, policy, quota, provider, or network issue. Fix the cause through a reviewed commit and generate a fresh plan.
6. Do not hand-edit Terraform-managed resources in the portal. For an approved emergency change, record it, notify the IaC owner, and reconcile configuration/state through a reviewed Terraform plan as soon as possible.

8. Post-deployment acceptance checklist

Complete and attach evidence to the change record:

- [ ] SQL logical server exists in the approved subscription, region and resource group; TLS minimum, Entra-only authentication, public network access, identity, tags, and Policy compliance match the design.
- [ ] Entra administrator object ID and tenant are correct. An authorized operator signs in with Entra authentication; no SQL administrator password has been created or shared.
- [ ] Every pool has the approved SKU, family, capacity, storage, per-database limits, and zone-redundancy setting. Every database is in the intended pool with correct maximum storage and retention.
- [ ] Private Endpoint is approved/connected. SQL FQDN resolves to the private address from each intended workload VNet and does not create an unintended public application path.
- [ ] From an approved workload host, TLS certificate validation succeeds and the application connects on TCP 1433 using its authorized identity. Denied connections from an unauthorized network are also verified.
- [ ] Diagnostic settings are enabled on the logical server, every elastic pool, and every database. Expected resource logs/metrics arrive in the approved workspace within the documented latency.
- [ ] CPU and data-I/O alerts target every pool with approved thresholds and

action group. Confirm delivery through an approved test; record responder acknowledgement and escalation path.

- [ ] Cost tags, budget alerts, Defender, auditing, retention, access reviews, and policy assignments meet the service's security/compliance baseline.
- [ ] Point-in-time restore is exercised into an isolated target and timed. Record the restore point, achieved RPO/RTO, owner, and cleanup approval.
- [ ] Regional recovery/failover is tested according to the approved plan; confirm secondary capacity, listener/DNS behavior, application reconnect, and data loss against RPO/RTO.
- [ ] SRE/DBA has dashboards, runbooks, on-call contacts, alert ownership, capacity review schedule, patch/change process, and escalation details.
- [ ] Terraform state is healthy, the pipeline is green, the artifact is access-restricted, and the next approved change can produce a no-op plan.

Do not onboard business data or declare production service until any failed acceptance item is resolved or formally risk-accepted by the accountable owner.

9. Operational monitoring and scaling

Review pool CPU and data I/O along with log-write, workers, sessions, storage, latency, errors, and database-level hotspots. Aggregate metrics can conceal a single overloaded database. Use Azure SQL Query Store/approved diagnostic queries to identify workload regressions without exporting data outside approved boundaries.

For sustained saturation, confirm the affected databases and correlated peak, check application/query changes and limits, evaluate pool isolation or right-sizing, calculate cost and recovery-region impact, test in nonproduction, then deploy via a reviewed plan. For storage pressure, investigate database growth and retention before resizing. Do not scale automatically from one transient metric or change pool configuration during an incident without the incident commander/change authority.

Review pool placement, database ownership, cost, alert thresholds, quota, and secondary capacity at the agreed cadence and after workload or business changes.

10. Incident and recovery actions

10.1 High utilization or widespread connection errors

1. Incident commander confirms impact, affected pool/database, scope, and severity; notify database and application owners.
2. Check Azure Service Health, resource health, pool/database metrics, diagnostic logs, Query Store, recent releases, connection throttling, workers, storage, and network/DNS from the workload subnet.
3. Mitigate through an approved application rollback, workload throttle, workload isolation, or pre-approved capacity change. Record cost and failover consequences. Use a fresh reviewed pipeline plan for IaC changes.
4. Verify recovery using application SLOs and alert recovery; preserve timeline, evidence, and follow-up actions.

10.2 Suspected data loss, corruption, or regional outage

1. Declare the incident, stop unsafe writes if directed, preserve evidence, and consult the service owner/DBA recovery decision authority.
2. Choose point-in-time restore, geo-restore, failover group, or geo-replication according to the approved data-loss and RTO decision. Confirm destination, credentials, capacity/quota, DNS/listener, encryption, and application dependencies.
3. Test recovered data and application invariants with the owner before cutover. Record actual restore point, data loss, duration, approvals, and endpoint change.

4. Restore the primary design only under a reviewed change plan. Reconcile Terraform and state without importing or replacing resources ad hoc.

Do not run failover, restore, import, state removal, or resource deletion from this template without the incident commander and authorized DBA approving the specific recovery action.

11. Change, teardown, and state safety

All infrastructure changes go through a reviewed PR, successful CI validation, protected branch, production plan review, and sql-prod approval. Refresh Terraform CLI/providers only through a reviewed dependency update and lock-file change. Keep state backups/version history and test restoration procedures.

There is no production destroy stage. Retiring a service requires a separate approved data-retention and backup/export plan, application cutover, owner sign-off, and a reviewed resource inventory. Do not delete or reinitialize state, unlock a lease, remove a resource from configuration, or use terraform state rm as an incident shortcut. Escalate lock/state recovery to the IaC platform owner and preserve forensic evidence.

12. Database schema and data onboarding

The Terraform database resource creates an empty pooled database. It does not create schemas, principals, application objects, or import/migrate business data. Treat database initialization and migration as a separate, versioned application release:

1. Confirm the database owner, source/target inventory, data classification, approved migration method, maintenance window, downtime/consistency requirements, and rollback/restore point.
2. Use the approved migration tool and a private network-connected migration runner. Authenticate with a managed identity or approved Entra principal; grant only the database permissions required for the migration and remove temporary grants after verification.
3. Version schema and seed/reference data changes in the application repository. Validate scripts against the target Azure SQL engine, selected service tier, collation, compatibility level, and application release.
4. Prefer backward-compatible expand/migrate/contract releases: add compatible objects first, deploy application code that can use both schemas, backfill in observable batches, validate row counts and business invariants, then remove obsolete objects only in a separately approved later release.
5. Test the full migration and recovery procedure against representative nonproduction data. Review lock duration, transaction/log growth, throttling, retry/idempotency behavior, and expected data loss/downtime.
6. On production, verify endpoint/private DNS and Entra access, take/confirm the required restore point, run the approved migration, validate application health and data invariants with the owner, and record timing/evidence.
7. If validation fails, stop writes/cutover as agreed and follow the approved restore or forward-fix decision. Do not assume a schema rollback reverses data changes, and do not run destructive rollback SQL without owner approval.

The platform pipeline deliberately has no SQL-password variable or application data migration command. Keep schema deployment separately reviewed and credentialed.

13. Ownership and review cadence

The service owner reviews objectives, risk acceptance, cost, and recovery. DBAs own pool/database placement, sizing, backups, and SQL health. Network owners own Private Link, DNS, routing, and address capacity. Identity/security owners approve authentication, federated identity, RBAC, policy, audit, and data controls. DevOps owns agent, pipeline, artifact, approvals, and branch

protection. SRE owns alert response, dashboards, incident process, and drills. Review this runbook after Azure service/provider changes, material architecture or compliance changes, every recovery exercise, and at least annually. Verify the current Microsoft provider/API guidance before production upgrades.